FIAP · FASE 6

Smart HAS
Oracle PL/SQL integrado ao ecossistema

Procedures e functions no banco, conectadas ao back-end Java, ao app Flutter e ao painel Angular por uma única API REST.



Giovanna Cardoso Pinheiro Cury Marques

RM 554433

GitHub: (inserir link)

Vídeo: (inserir link do YouTube — não listado)

CONTEXTO E OBJETIVOS

Da API com H2 à inteligência no banco

Fase 5 (MVP)

Flutter + Spring Boot + Angular

API REST com JWT

H2 em memória: CRUD de sensores e alertas

Sem histórico, sem regras automáticas

Objetivo da Fase 6

Persistência Oracle com modelo relacional

Regras de monitoramento em PL/SQL

Procedure disparada por evento do back-end

REST → Java → JDBC → Oracle

Resultado

6 tabelas · 4 functions · 5 procedures

~2.700 leituras simuladas (7 dias)

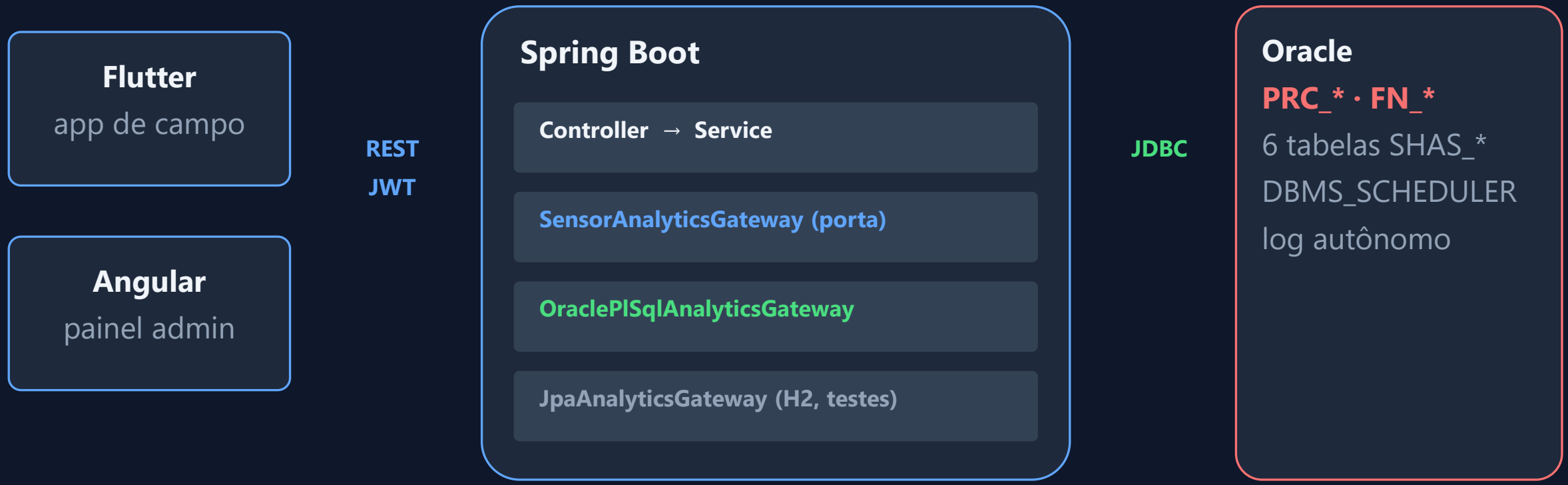
Alertas automáticos + relatórios + watchdog

25 testes H2 + 4 testes no Oracle real

Fase 6: a inteligência de monitoramento (classificar, alertar, medir, consolidar, vigiar) passa a rodar no Oracle, reutilizável pelo app, pelo painel e por jobs agendados.

INTEGRAÇÃO ENTRE CAMADAS

Arquitetura: a regra vive no banco, o Java orquestra



Ports & Adapters: o perfil Spring escolhe o adapter. Com 'oracle', as regras executam em PL/SQL (Hibernate só valida o schema); sem Oracle, o mesmo contrato roda em Java para demo e testes.

APRIMORAMENTO DA SOLUÇÃO

Parte 1 — O que evoluiu

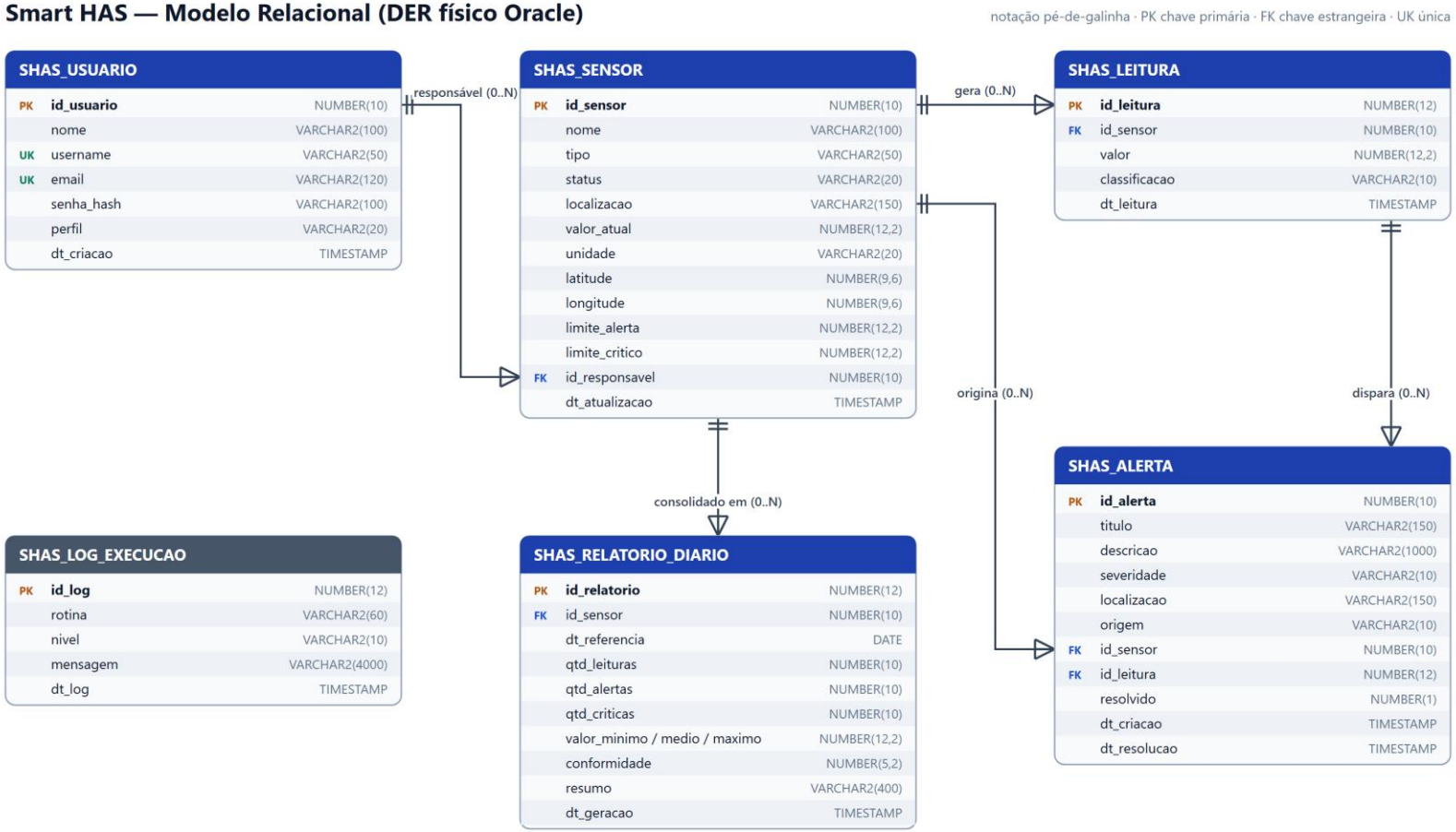
Novas funcionalidades

- Telemetria: registro e histórico de leituras
- Classificação NORMAL / ALERTA / CRÍTICA por limites
- Alertas automáticos com deduplicação e escalonamento
- Indicadores: conformidade 24h/7d e risco por local
- Relatório diário, resumo por responsável, auditoria
- Watchdog de sensores inativos + simulador IoT

Refatoração e boas práticas

- Ports & Adapters (gateway por perfil)
- Schema versionado em SQL + ddl-auto=validate
- ORA-20001 → 404 · ORA-20002..04 → 422
- Transação online x batch explícita
- Contrato REST retrocompatível
- Angular: KPIs do back-end, pt-BR, design system
- Flutter: fallback central + --dart-define

Parte 2 — Modelo relacional Oracle



Implantação

instalar_tudo.sql (reexecutável)

Docker: Oracle 23ai Free, instala sozinho

SQL Developer: servidor FIAP (19c)

Constraints nomeadas + COMMENT

Carga PL/SQL reprodutível:

3 usuários · 8 sensores

2.685 leituras · 40 alertas

64 linhas de relatório

PL/SQL

Parte 3 — Functions: indicadores e dados formatados

FN_CALC_CONFORMIDADE

Indicador · % de leituras normais na janela (24h, 7d). NULL sem dados.

FN_CALC_INDICE_RISCO

Indicador · 0–100 por local: alertas ponderados + sensores degradados.

FN_FORMATAR_LEITURA

Dados formatados · “Sensor 03 (Qualidade do Ar) | 162,40 AQI | CRITICA | 27/09 20:20”

FN_CLASSIFICAR_LEITURA

Regra (DETERMINISTIC) · NORMAL / ALERTA / CRITICA, reutilizada pelas procedures.

```
SELECT localizacao, fn_calc_indice_risco(localizacao) AS risco
  FROM (SELECT DISTINCT localizacao FROM shas_sensor) ORDER BY 2 DESC;   → Marginal Tietê 75 · Ibirapuera 50
Boas práticas: IN/RETURN com %TYPE · RAISE_APPLICATION_ERROR · NO_DATA_FOUND tratado (evita NULL silencioso)
```

PL/SQL

Parte 3 — Procedures: rotinas de back-end no banco

PRC_REGISTRAR_LEITURA
Online · chamada pelo Java

Valida → FOR UPDATE → classifica → grava → atualiza sensor → cria / deduplica / escalona alerta. SAVEPOINT + EXCEPTION.

PRC_GERAR_RELATORIO_DIARIO
Batch · job 23:55

CURSOR + FOR LOOP + MERGE idempotente, usando as functions.

PRC_VERIFICAR_SENSORES_INATIVOS
Batch · job 15 min

OPEN/FETCH/CLOSE, WHERE CURRENT OF: marca OFFLINE e abre alerta.

PRC_RESUMO_POR_RESPONSAVEL
Consulta · SYS_REFCURSOR

Resumo por usuário, lido no Java como ResultSet.

+ PRC_REGISTRAR_LOG (AUTONOMOUS_TRANSACTION): toda execução e todo erro ficam auditados, mesmo após ROLLBACK.

INTEGRAÇÃO COM O BACK-END

REST → Java → JDBC → Oracle

```
new SimpleJdbcCall(jdbcTemplate)
  .withProcedureName("PRC_REGISTRAR_LEITURA")
  .withoutProcedureColumnMetaDataAccess()
  .withNamedBinding()   // p_x => ?
  .declareParameters(
     IN  p_id_sensor, p_valor, p_dt_leitura
     OUT p_id_leitura, p_classificacao,
         p_id_alerta);

// SYS_REFCURSOR
new SqlOutParameter("p_cursor",
    OracleTypes.CURSOR, RESPONSIBLE_MAPPER)
```

Endpoints → Oracle

POST /sensors/{id}/readings → PRC_REGISTRAR_LEITURA

GET /sensors/{id}/indicators → 3 functions

GET /reports/responsibles → ref cursor

POST /routines/* (ADMIN) → rotinas batch

ORA-20001 → HTTP 404

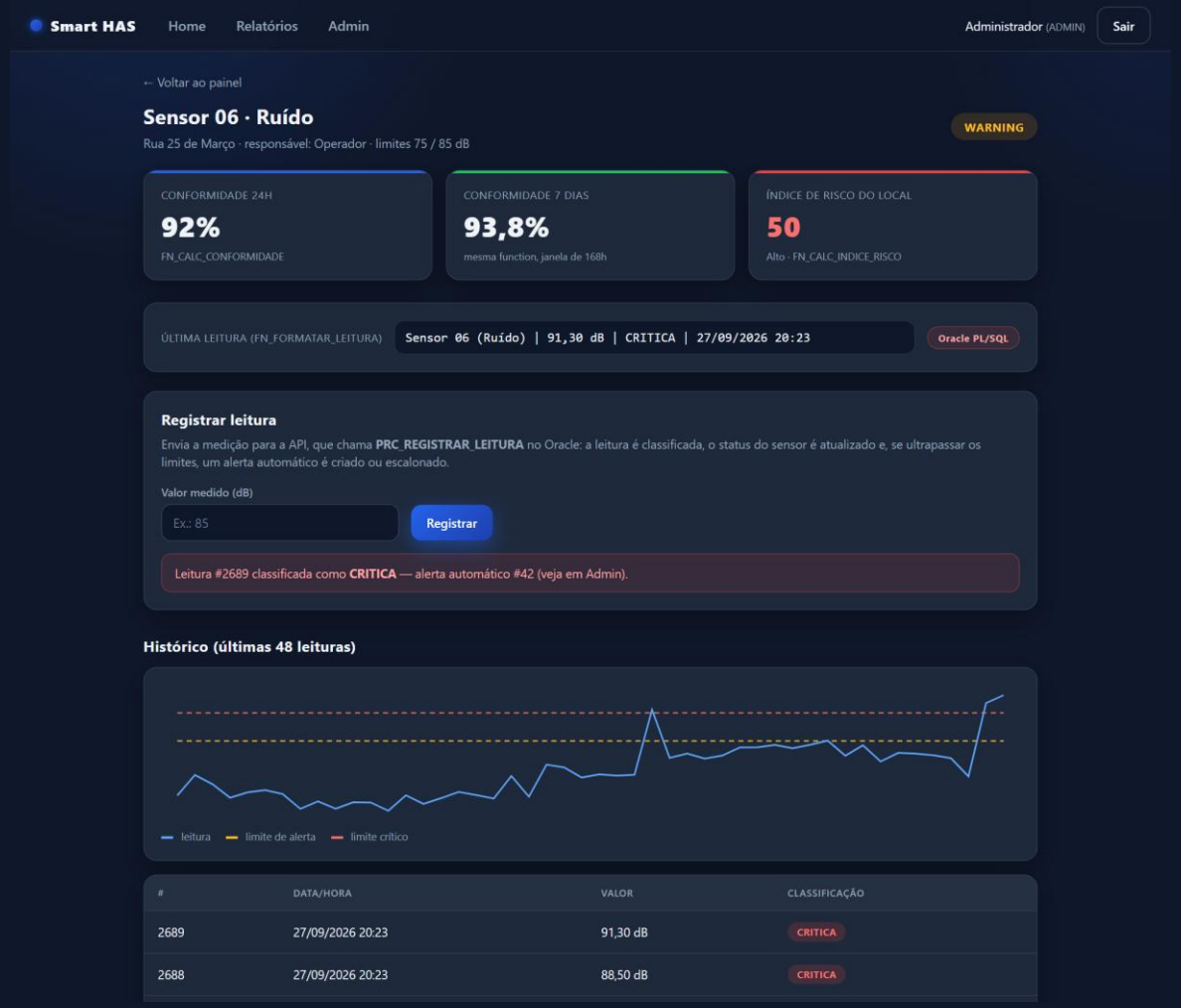
ORA-20002..04 → HTTP 422

Transação: @Transactional (online) · COMMIT na procedure (batch)

Evento de back-end: cada leitura do app, do painel ou do simulador IoT (@Scheduled) dispara a procedure. Validado por teste de integração contra o Oracle real.

APLICAÇÃO RODANDO

Demonstração: leitura crítica vira alerta automático





1. Registrar 91,3 dB
   → **CRITICA, alerta #42**

2. Risco do local sobe para 50

3. Relatório, ref cursor e auditoria no painel

CONCLUSÃO

Resultados e próximos passos

6	4	5	2.685	29
tabelas	functions	procedures	leituras	testes

Valor entregue

Regra única para app, painel e jobs
Alertas sem intervenção humana
KPIs de qualidade e risco para decisão
Auditoria e reprocessamento (governança)

Próximos passos

Deploy em nuvem (Oracle Autonomous + HTTPS)
Push (FCM) para alertas críticos
Camada de IA sobre a série histórica
Particionamento de SHAS_LEITURA por data